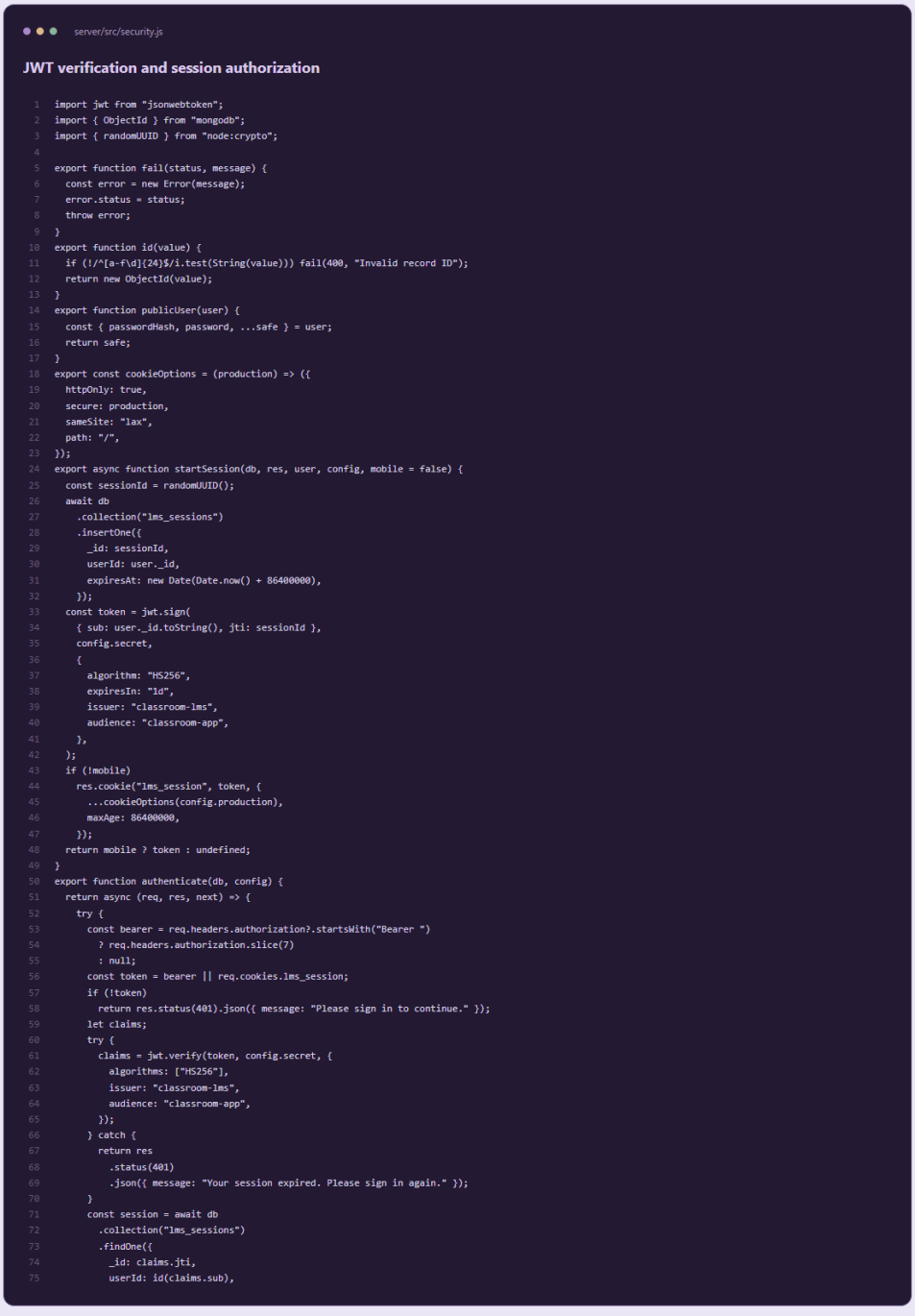
Source Code Screenshots

Selected implementation excerpts demonstrating authentication, validation, authorization, file handling, React hooks/routing, and Flutter API/submission behavior.

The complete editable source accompanies these screenshots. Secrets and environment configuration are excluded.

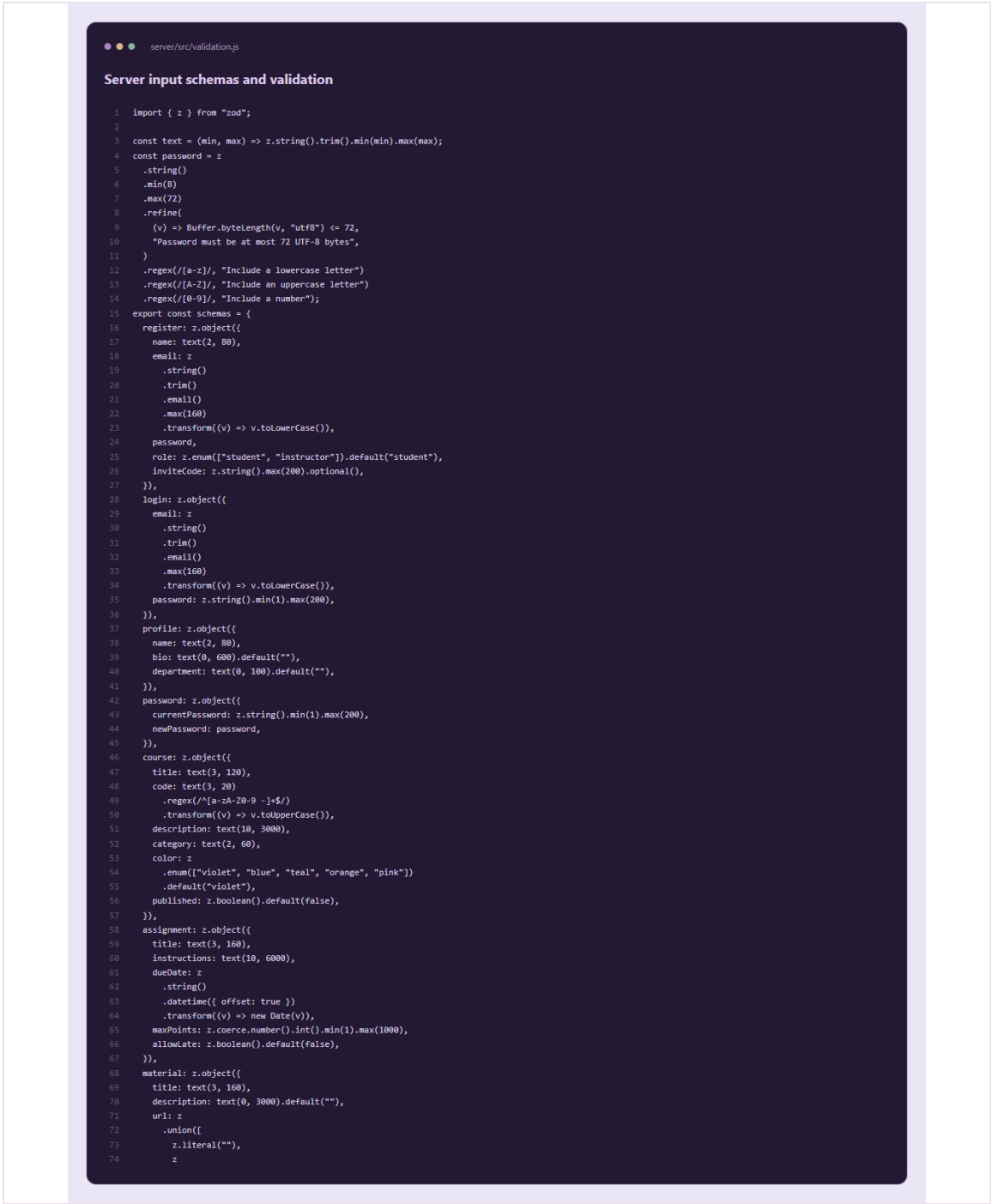
1. JWT verification and session authorization

server/src/security.js



2. Server input schemas and validation

server/src/validation.js



3. Course ownership and enrollment checks

server/src/courses.js

server/src/courses.js

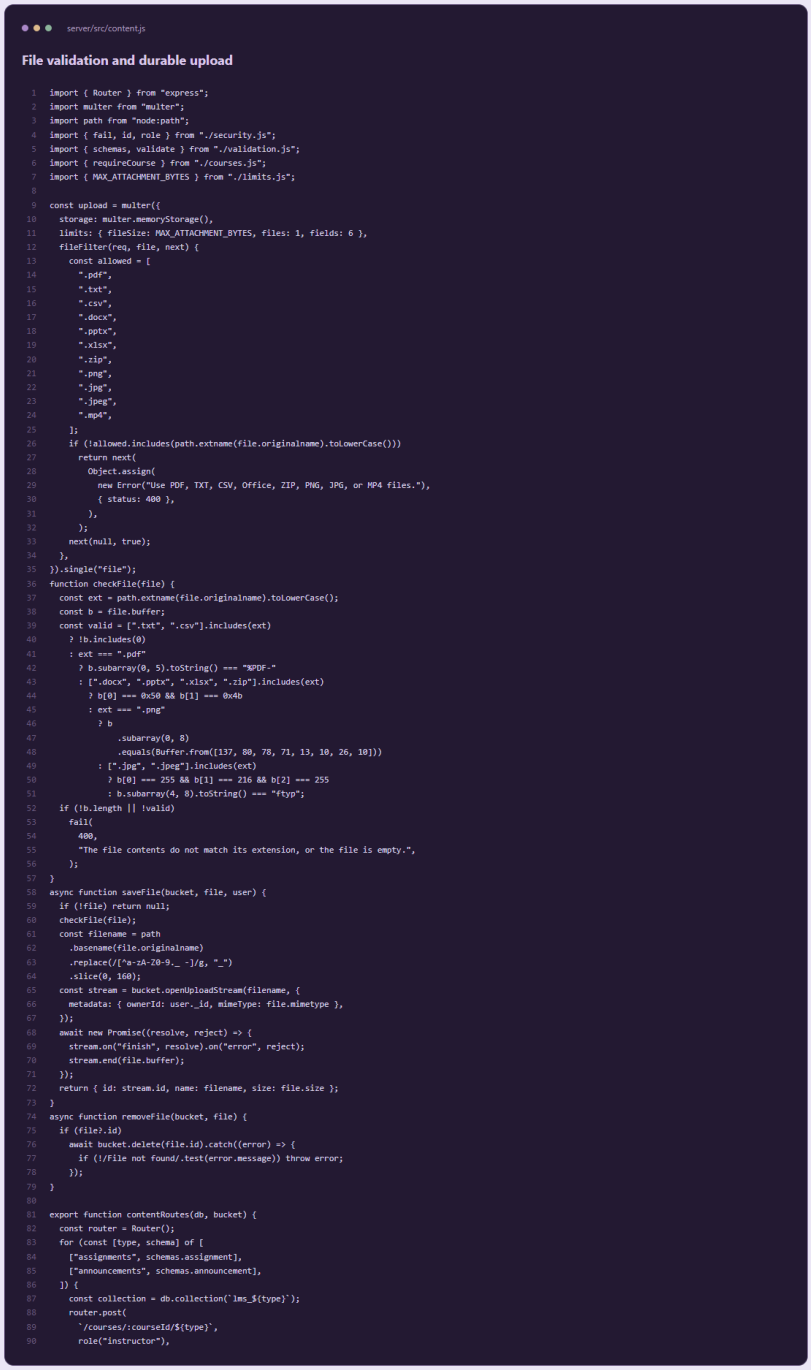
Course ownership and enrollment checks

```
import { Router } from "express";
import { fail, id, role } from "./security.js";
import { schemas, validate } from "./validation.js";

export async function accessCourse(db, user, courseId, owner = false) {
  const course = await db
    .collection("lms_courses")
    .findOne({ _id: id(courseId), deletedAt: { $exists: false } });
  if (!course) fail(404, "Course not found.");
  const isOwner = course.instructorId.equals(user._id);
  if (owner && !isOwner)
    fail(403, "Only the instructor who owns this course can change it.");
  const enrolled =
    user.role === "student" &&
    !!(await db
      .collection("lms_enrollments")
      .findOne({ courseId: course._id, studentId: user._id }));
  return { course, isOwner, enrolled };
}
export async function requireCourse(db, user, courseId, owner = false) {
  const result = await accessCourse(db, user, courseId, owner);
  if (!result.isOwner && !result.enrolled)
    fail(403, "Join this course to access its learning content.");
  return result;
}
export async function visibleCourseIds(db, user) {
  if (user.role === "instructor")
    return (
      await db
        .collection("lms_courses")
        .find({ instructorId: user._id, deletedAt: { $exists: false } })
        .project({ _id: 1 })
        .toArray()
    ).map((c) => c._id);
  const enrolled = await db
    .collection("lms_enrollments")
    .find({ studentId: user._id })
    .toArray();
  return (
    await db
      .collection("lms_courses")
      .find({
        _id: { $in: enrolled.map((e) => e.courseId) },
        deletedAt: { $exists: false },
      })
      .project({ _id: 1 })
      .toArray()
  ).map((c) => c._id);
}
export function courseRoutes(db, bucket) {
  const router = Router();
  const courses = db.collection("lms_courses");
  const enrollments = db.collection("lms_enrollments");
  router.get("/courses", async (req, res) => {
    const page = Math.max(1, Number.parseInt(req.query.page) || 1);
    const limit = 9;
    const search =
      typeof req.query.search === "string"
        ? req.query.search.slice(0, 100).replace(/[.*+?^${}()|[\]\\]/g, "\\$&")
        : "";
    const filter = {
      deletedAt: { $exists: false },
      ...(req.user.role === "instructor"
        ? { instructorId: req.user._id }
        : req.query.scope === "catalog"
          ? { published: true }
          : { _id: { $in: await visibleCourseIds(db, req.user) } }),
    };
    if (search)
      filter.$or = [
        { title: { $regex: search, $options: "i" } },
        { code: { $regex: search, $options: "i" } },
        { category: { $regex: search, $options: "i" } },
```

4. File validation and durable upload

server/src/content.js



5. Protected React routing

client/src/App.jsx

client/src/App.jsx

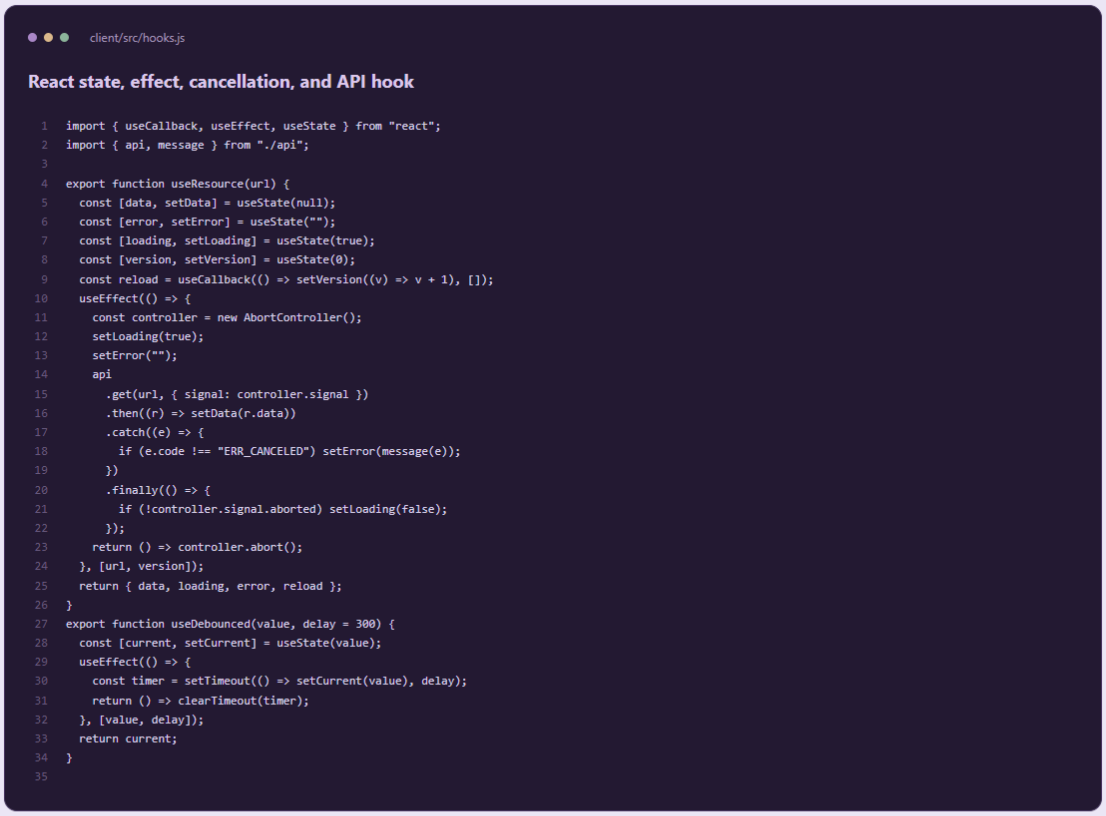
Protected React routing

```jsx
import { BrowserRouter, Navigate, Route, Routes } from "react-router-dom";
import { Toaster } from "react-hot-toast";
import { AuthProvider, useAuth } from "./auth";
import AuthPage from "./AuthPage";
import Shell from "./Shell";
import Dashboard from "./Dashboard";
import Courses from "./Courses";
import CourseDetail from "./CourseDetail";
import ActivityPage from "./ActivityPage";
import Profile from "./Profile";
import { ErrorState, Loading } from "./ui";
function Protected() {
  const { user, loading, unavailable, restore } = useAuth();
  if (loading) return <Loading />;
  if (unavailable)
    return (
      <ErrorState
        error="The server is unavailable. Start the API and check your database connection."
        retry={restore}
      />
    );
  return user ? <Shell /> : <Navigate to="/login" replace />;
}
export default function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <Routes>
          <Route path="/login" element={<AuthPage />} />
          <Route path="/register" element={<AuthPage register />} />
          <Route path="/signup" element={<Navigate to="/register" replace />} />
          <Route element={<Protected />}>
            <Route index element={<Dashboard />} />
            <Route path="/courses" element={<Courses />} />
            <Route path="/courses/:courseId" element={<CourseDetail />} />
            <Route
              path="/assignments"
              element={<ActivityPage type="assignments" />}
            />
            <Route path="/grades" element={<ActivityPage type="grades" />} />
            <Route
              path="/announcements"
              element={<ActivityPage type="announcements" />}
            />
            <Route path="/profile" element={<Profile />} />
            <Route path="*" element={<Navigate to="/" replace />} />
          </Route>
        </Routes>
      </BrowserRouter>
      <Toaster
        position="top-right"
        toastOptions={{
          duration: 4000,
          style: { fontSize: 14, borderRadius: 12 },
        }}
      />
    </AuthProvider>
  );
}

```

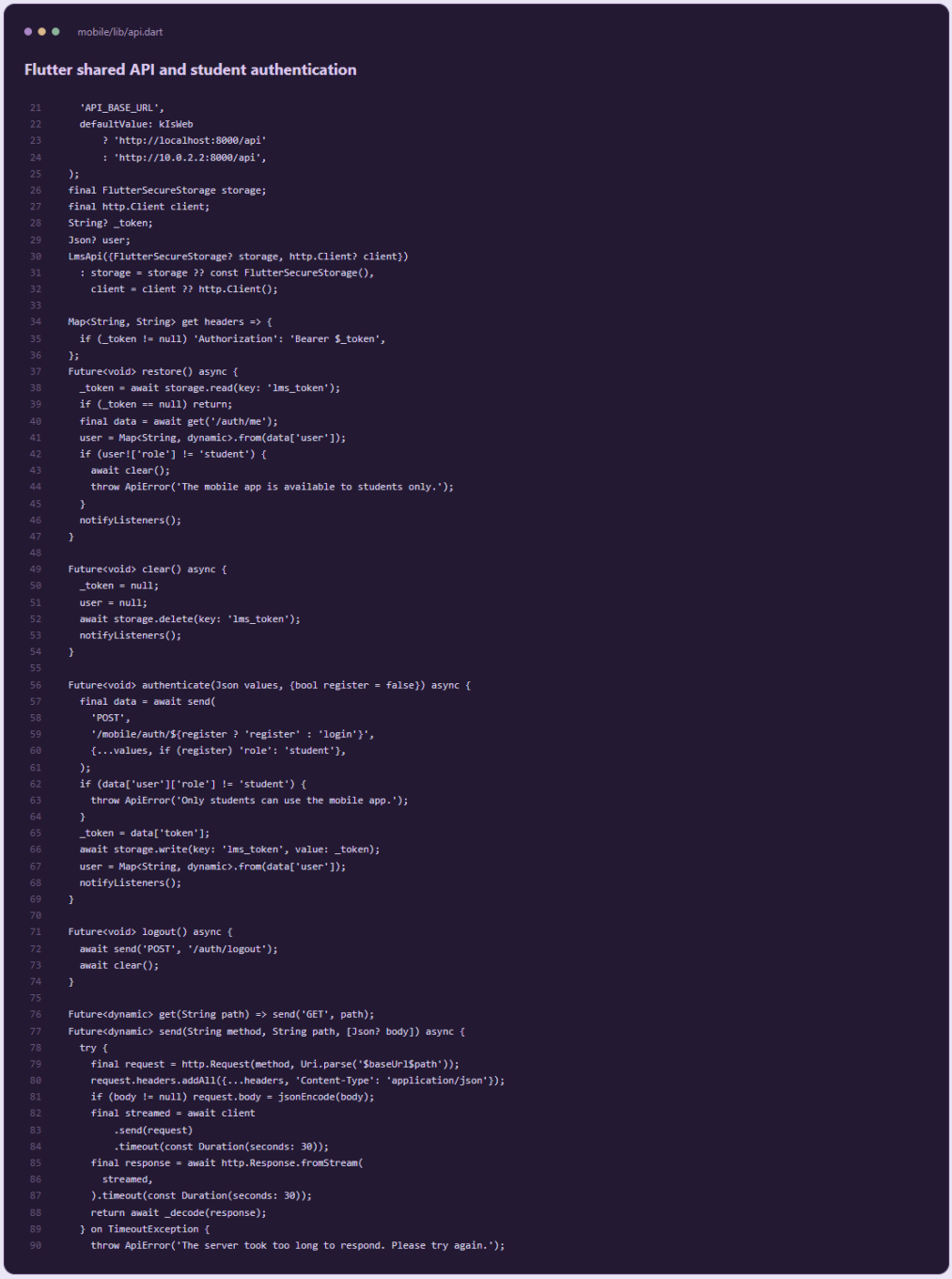
6. React state, effect, cancellation, and API hook

client/src/hooks.js



7. Flutter shared API and student authentication

mobile/lib/api.dart



8. Flutter assignment submission and confirmation

mobile/lib/submission_screen.dart

